Coeficiente de Variação com OpenMP

Exercício Final – Análise de Dados com OpenMP · Ciência da Computação – IESB
Prof. Rodrigo · Aluna: Paula Ribeiro Moreira de Souza

1. Visão geral

O programa `cv_openmp.c` lê o arquivo `dados/dados.txt` (1000 linhas, cada uma com estatura em cm e peso em kg separados por espaço), agrupa cada variável em intervalos de classe (8 cm para estatura e 4 kg para peso) e calcula a média ponderada, o desvio padrão populacional e o coeficiente de variação. Como não foi fornecido um arquivo de dados oficial, foi usado um conjunto fictício de 1000 pessoas com valores realistas. O cálculo foi dividido em quatro etapas, cada uma em uma função com seu próprio laço: (1) mínimo e máximo, (2) distribuição de frequências, (3) média ponderada e (4) desvio padrão. A primeira classe começa no piso do menor valor observado, e o número de classes é $k = \lfloor (\text{máx} - \text{lim}_{\text{inf}}) / h \rfloor + 1$, de modo que todos os dados ficam dentro de alguma classe.

2. Partes paralelizadas e por quê

Todos os laços usam `#pragma omp parallel for` com uma cláusula `if(paralelo)`. Assim, o mesmo código roda nas versões sequencial (`paralelo = 0`, sem criar equipe de threads) e paralela, o que torna a comparação de tempo justa. Em todos os casos a sincronização é feita por redução (`reduction`): cada thread acumula em uma variável privada e o OpenMP combina os resultados no final, evitando condições de corrida sem usar `critical` ou `atomic`, que serializariam o acesso.

Etapa	Laço sobre	Diretiva OpenMP	Justificativa
Mínimo e máximo	N dados	reduction(min:lo) reduction(max:hi)	Iterações independentes; cada thread acha o mín/máx do seu bloco.
Frequências	N dados	reduction(+:freq[:k])	Etapa mais custosa. Redução de vetor: cada thread conta em uma cópia privada de freq[].
Média ponderada	k classes	reduction(+:soma_xf, soma_f)	Soma de $x_i \cdot f_i$ e de f_i, conforme a fórmula (1).
Desvio padrão	k classes	reduction(+:soma)	Soma de $f_i(x_i - \mu)^2$, conforme a fórmula (2).

Os laços de média e desvio padrão percorrem apenas as k classes (7 para estatura e 18 para peso), não as 1000 pessoas. Eles foram paralelizados porque o enunciado pede, mas com tão poucas iterações o ganho é desprezível; o trabalho que realmente escala com o tamanho da entrada está nos laços de mínimo/máximo e de frequências. O trecho principal da contagem de frequências é:

```
#pragma omp parallel for if(paralelo) reduction(+:freq[:k])
for (long i = 0; i < n; i++) {
    int c = (int)((v[i] - lim_inf) / h);   /* classe do dado i */
    if (c >= k) c = k - 1;
    freq[c]++;                             /* cópia privada    */
}
```

3. Resultados

Variável	h	Classes (k)	Média μ	Desvio σ	CV
Estatura	8 cm	7	170,38 cm	9,16 cm	**5,37 %**
Peso	4 kg	18	70,07 kg	12,67 kg	**18,08 %**

As tabelas de frequências completas são impressas pelo programa ao ser executado. O peso tem variabilidade relativa cerca de 3,4 vezes maior que a estatura. Isso é esperado: a estatura das pessoas varia pouco em torno da média, enquanto o peso depende de vários fatores além da altura. Como conferência, o CV calculado direto dos dados brutos (sem agrupamento) é 5,30

% para estatura e 18,08 % para peso, muito próximo dos valores agrupados; a diferença vem da aproximação de cada dado pelo ponto médio da sua classe.

4. Análise de desempenho

O tempo foi medido com `omp_get_wtime()`. Como uma execução com N = 1000 leva poucos microssegundos, o programa repete o cálculo completo das duas variáveis 2000 vezes e reporta a média, após uma execução de aquecimento. Para observar o comportamento em escala, o mesmo código foi executado também com 10 milhões de registros gerados em memória (`./cv_openmp --bench 10000000`, média de 5 execuções).

Ambiente de teste. GitHub Codespaces, processador AMD EPYC 7763, compilado com `gcc -O2 -fopenmp`. O comando `nproc` informa 2 processadores, mas `lscpu` mostra *1 núcleo físico com 2 threads de hardware* (Thread(s) per core: 2; Core(s) per socket: 1). Ou seja, as duas threads do OpenMP compartilham as mesmas unidades de execução por SMT (hyperthreading), e não há dois núcleos independentes.

Versão	Threads	N = 1000 (ms)	Speedup	N = 10 mi (ms)	Speedup
Sequencial	1	0,0148	1,00×	50,80	1,00×
Paralela	1	0,0118	1,26×	45,08	1,13×
Paralela	2	0,0196	0,75×	56,09	0,91×

N = 1000. Com 2 threads a versão paralela ficou mais lenta que a sequencial (0,75×). O trabalho total é minúsculo (alguns milhares de operações), e o custo de criar e sincronizar a equipe de threads e combinar as reduções a cada região paralela é maior que o próprio cálculo. Para esse tamanho de entrada, o overhead do OpenMP domina, e a versão sequencial é a melhor escolha.

N = 10 milhões. Com volume grande, o custo fixo do paralelismo se torna proporcionalmente pequeno e o tempo passa a ser dominado pelos laços sobre os dados (mínimo/máximo e frequências). Mesmo assim, 2 threads foram 9 % mais lentas que a versão sequencial. Dois fatores explicam isso. Primeiro, como a máquina tem um único núcleo físico, as duas threads disputam o mesmo núcleo: o SMT só ajuda quando uma thread fica parada esperando e a outra aproveita as unidades livres, e aqui as duas executam o mesmo tipo de instrução. Segundo, esses laços fazem pouca conta por elemento e passam a maior parte do tempo lendo os 80 MB de cada vetor da memória; um laço limitado pela banda de memória ganha pouco com mais threads no mesmo núcleo. Como conferência de corretude em escala, os CVs dos dados sintéticos (gerados com média 170 e desvio 9 para estatura, média 70 e desvio 12 para peso) foram 5,47 % e 17,22 %, compatíveis com os valores teóricos de 5,29 % e 17,14 %; a pequena diferença vem do agrupamento em classes, que acrescenta cerca de $h^2/12$ à variância.

Paralela com 1 thread. A versão paralela com uma única thread ficou um pouco mais rápida que a sequencial nos dois casos (1,26× e 1,13×), o que não reflete paralelismo real. A explicação mais provável é que o compilador gera código diferente para o corpo da região paralela (que o OpenMP transforma em uma função separada), somado à variação natural das medições em uma máquina virtual compartilhada.

Expectativa em uma máquina com vários núcleos. Com núcleos físicos independentes, os laços sobre os dados podem de fato ser divididos, e a expectativa é de speedup crescente com o número de threads para N grande, limitado pela parte que continua sequencial (leitura do arquivo, geração dos dados e laços sobre as poucas classes), como prevê a Lei de Amdahl, e pela banda de memória.

Conclusão. O programa produz os mesmos resultados nas duas versões e está corretamente paralelizado com reduções. O ganho de desempenho do OpenMP depende do volume de dados e do hardware: para as 1000 pessoas do exercício o overhead supera o benefício, e no ambiente testado, com um único núcleo físico, nem o volume de 10 milhões de registros produziu ganho. Em uma máquina com vários núcleos, os laços sobre os dados são os candidatos naturais à paralelização.